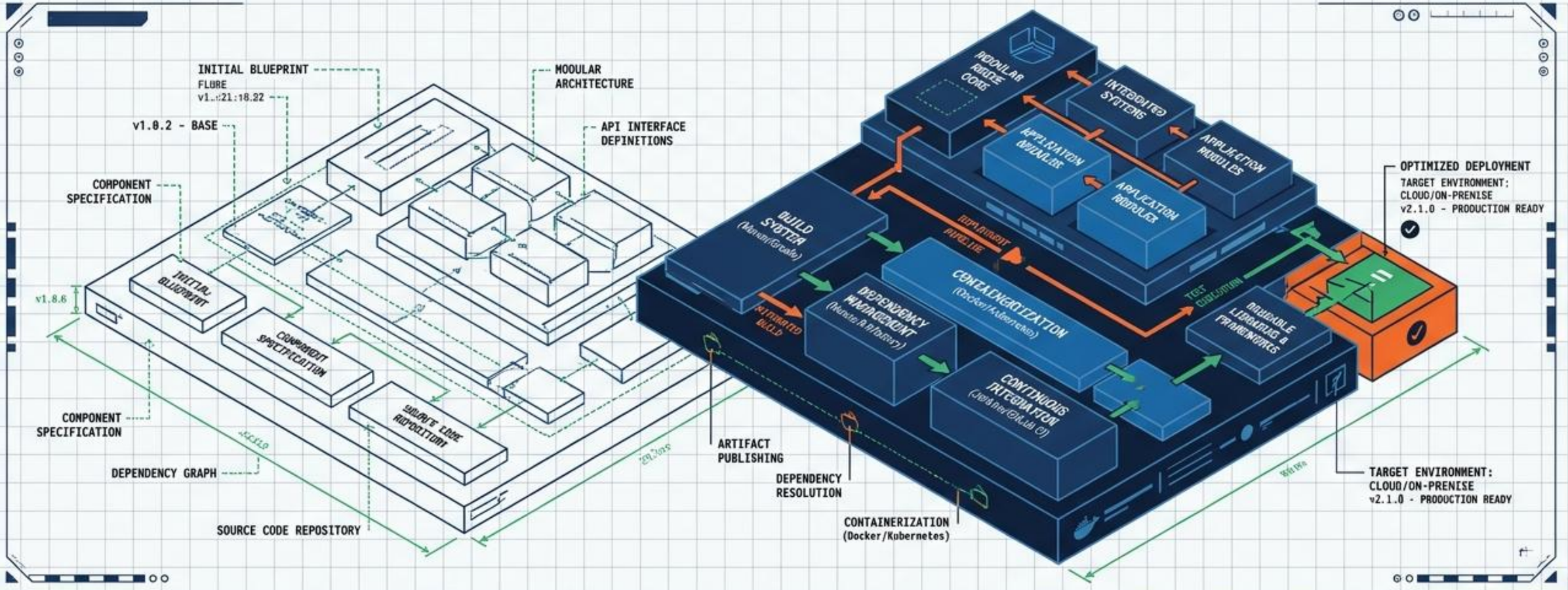
Universidad Nacional de Ingeniería (UNI) – EPIS
SW-605 Construcción de Software II | 7mo Ciclo

Semana 1 — Introducción a las Tecnologías de Construcción, Reúso y Dependencias



ERGINEERING SCHENATIC // SYSTEN ARCHITECTURE v1.4 // DATE: OCT 26, 2024 // APPROVED FOR DEVELOPMENT

Gemini Notebook

De la Sintaxis a la Realidad en Ejecución

La construcción avanzada trasciende la codificación. El enfoque evoluciona de diseñar estructuras estáticas a dominar el comportamiento dinámico.

Sintaxis Estática

STATIC STRUCTURE

```
const StaticDefinition = {
  id: "static-001",
  type: "EntityModel",
  attributes: {
    status: "INACTIVE",
    dataPoints: 1024,
    configuration: "DEFAULT"
  },
  methods: [
    "initialize",
    "validate",
    "process"
  ],
  dependencies: [
    "CoreLib",
    "DataParser"
};
// End of Static Definition
```

CODE BLOCK DEFINITION

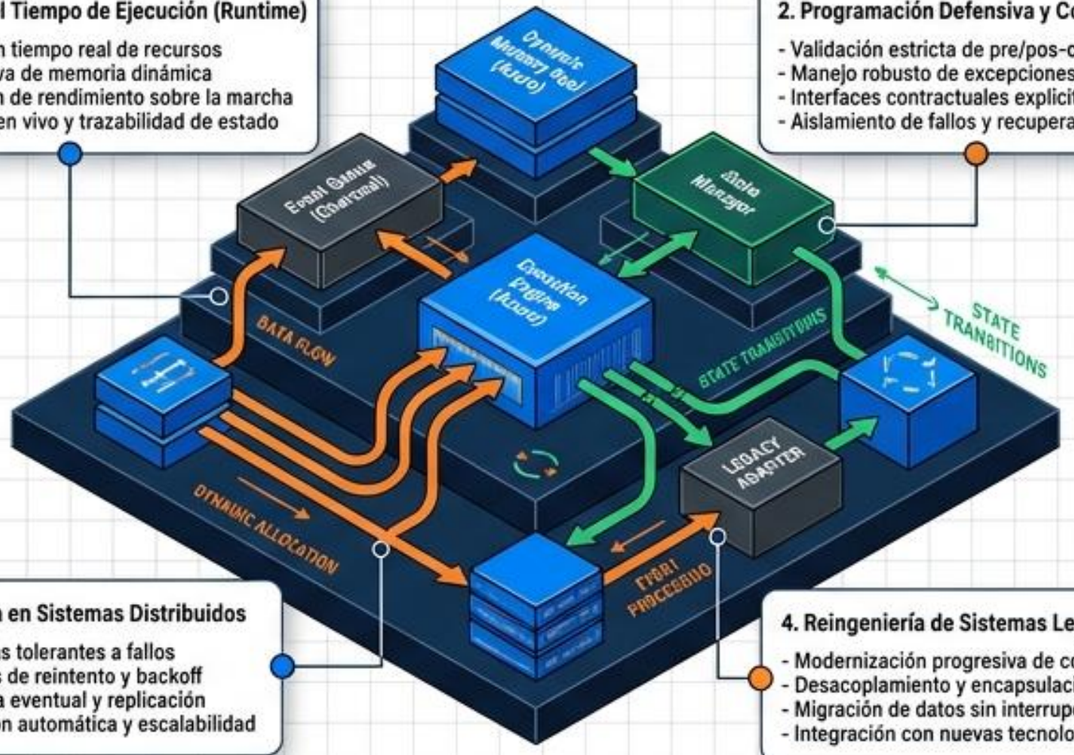
Realidad en Ejecución

1. Dominio del Tiempo de Ejecución (Runtime)

- Monitoreo en tiempo real de recursos
- Gestión activa de memoria dinámica
- Optimización de rendimiento sobre la marcha
- Depuración en vivo y trazabilidad de estado

2. Programación Defensiva y Contratos

- Validación estricta de pre/pos-condiciones
- Manejo robusto de excepciones
- Interfaces contractuales explícitas
- Aislamiento de fallos y recuperación



3. Resiliencia en Sistemas Distribuidos

- Arquitecturas tolerantes a fallos
- Mecanismos de reintento y backoff
- Consistencia eventual y replicación
- Recuperación automática y escalabilidad

4. Reingeniería de Sistemas Legacy

- Modernización progresiva de componentes
- Desacoplamiento y encapsulación
- Migración de datos sin interrupción
- Integración con nuevas tecnologías

Gemini Notebook

Delimitación Conceptual: Fronteras Entrelazadas



Insight: La construcción moderna no es una fase aislada; actúa como el tejido conectivo que entrelaza el diseño con la integración verificable.

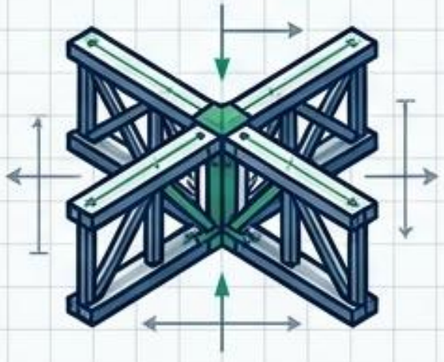
¿Qué es la Construcción de Software?

> “El proceso de creación de software funcional a través de una combinación detallada de codificación, verificación, pruebas unitarias, depuración y optimización.”



Traducción Lógica

Convertir modelos humanos abstractos en instrucciones de máquina predecibles y estrictas.



Ingeniería Estructural

Ensamblaje seguro considerando la memoria, el rendimiento y la concurrencia a nivel de sistema.



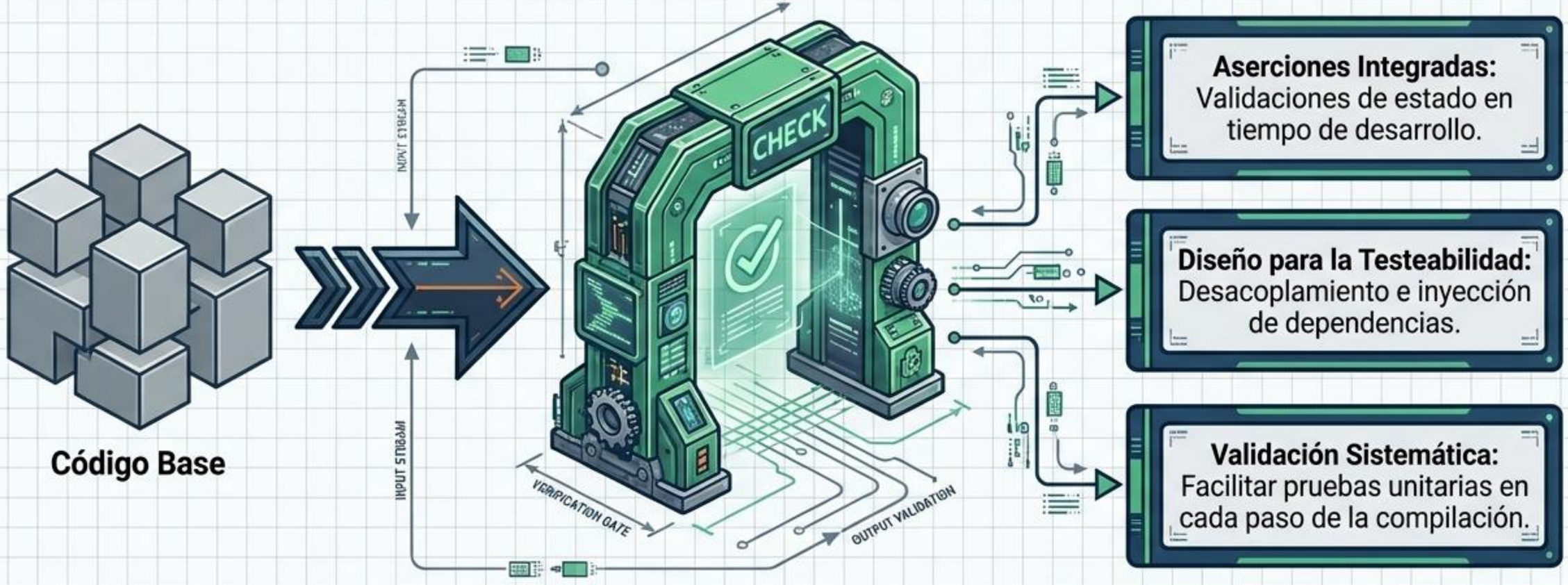
Mantenibilidad

Escribir código para el ingeniero del mañana y para la resiliencia del sistema a largo plazo.

Gemini Notebook

Construcción para la Verificación

Premisa: Escribir código asumiendo que debe ser probado y validado continuamente. El código no verificable es código frágil.



Código Base

Aserciones Integradas: Validaciones de estado en tiempo de desarrollo.

Diseño para la Testeabilidad: Desacoplamiento e inyección de dependencias.

Validación Sistemática: Facilitar pruebas unitarias en cada paso de la compilación.

© 2024 Premium Engineeri...
Gemini Notebook

El Fantasma en la Máquina: Tiempo de Ejecución (Runtime)

El Eje de SW-605: El código fuente es predecible y estático; el software en ejecución no lo es. El comportamiento cambia dinámicamente basándose en el estado, la concurrencia y el entorno.

Enlace Dinámico (Dynamic Binding)

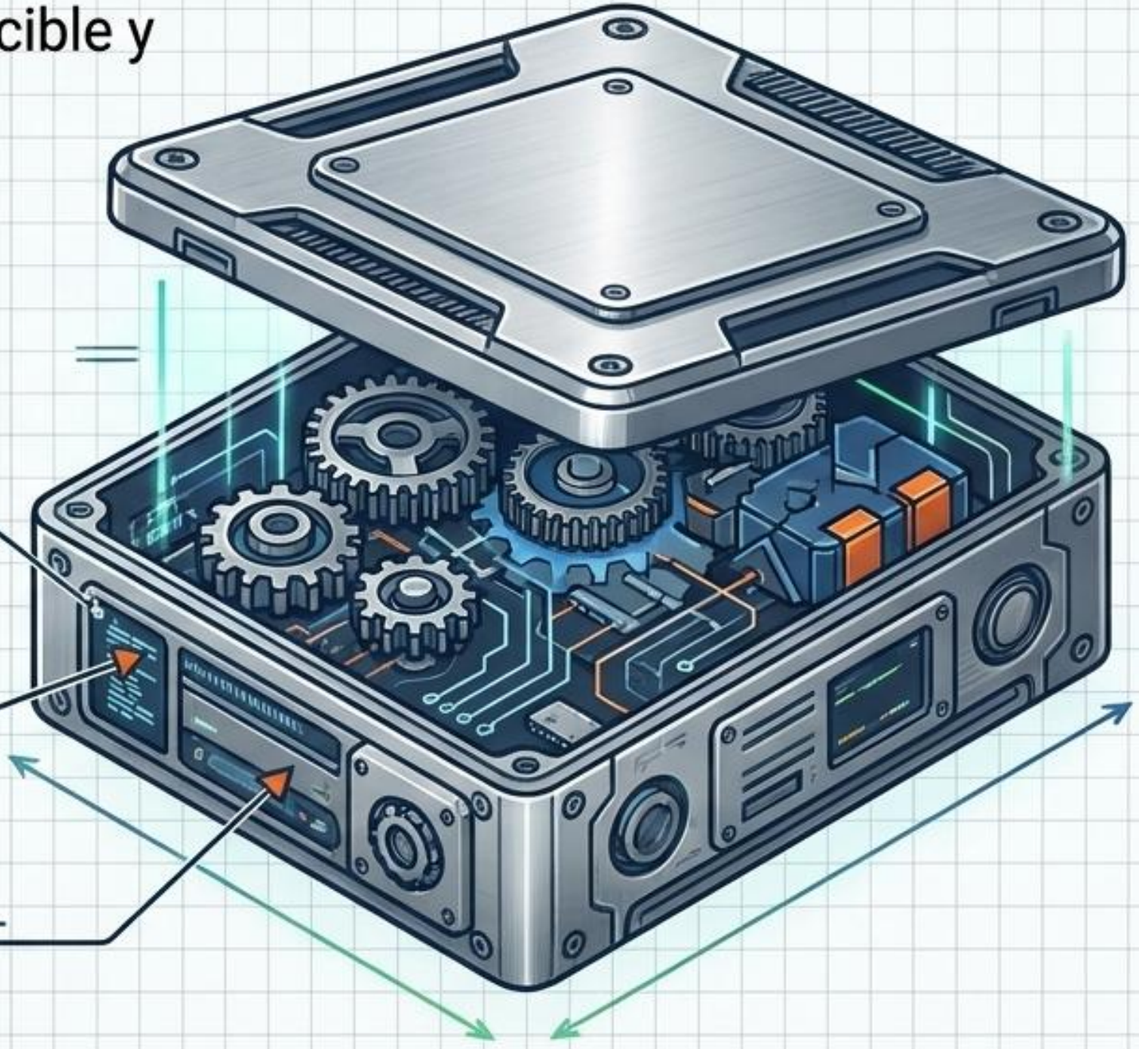
Resolución de métodos y variables en tiempo de ejecución en lugar de compilación.

Reflexión (Reflection)

Capacidad del programa para examinar y modificar su propia estructura y comportamiento.

Genéricos (Generics / Templates)

Flexibilidad en tipos de datos, optimizando la reutilización y el rendimiento en ejecución.



© 2024 Premium Engineeri...
Gemini Notebook

Anatomía de los Mecanismos Dinámicos

RUNTIME RESOLUTION: <1ms



RUNTIME RESOLUTION: <1ms

1. Enlace Dinámico

Resolviendo llamadas a métodos en el último milisegundo. Polimorfismo en acción real; el compilador cede el control a la máquina virtual basándose en el tipo real del objeto en memoria.

CODE INSPECTION: ENABLED



CODE INSPECTION:
ENABLED

METAPROGRAMMING: ACTIVE

2. Reflexión y Metaprogramación

El código que se inspecciona, invoca y modifica a sí mismo durante la ejecución. Un mecanismo extremadamente potente para frameworks, pero riesgoso para la seguridad y el rendimiento.

TYPE SAFETY: GUARANTEED



TYPE SAFETY: GUARANTEED
STATIC VERIFICATION: PASS

3. Genéricos

Composición de comportamiento seguro en tipos durante la ejecución. Permite la parametrización de clases y métodos sin sacrificar el rendimiento ni la verificación estática del compilador.

© 2024 Premium Engineering Systems

Gemini Notebook

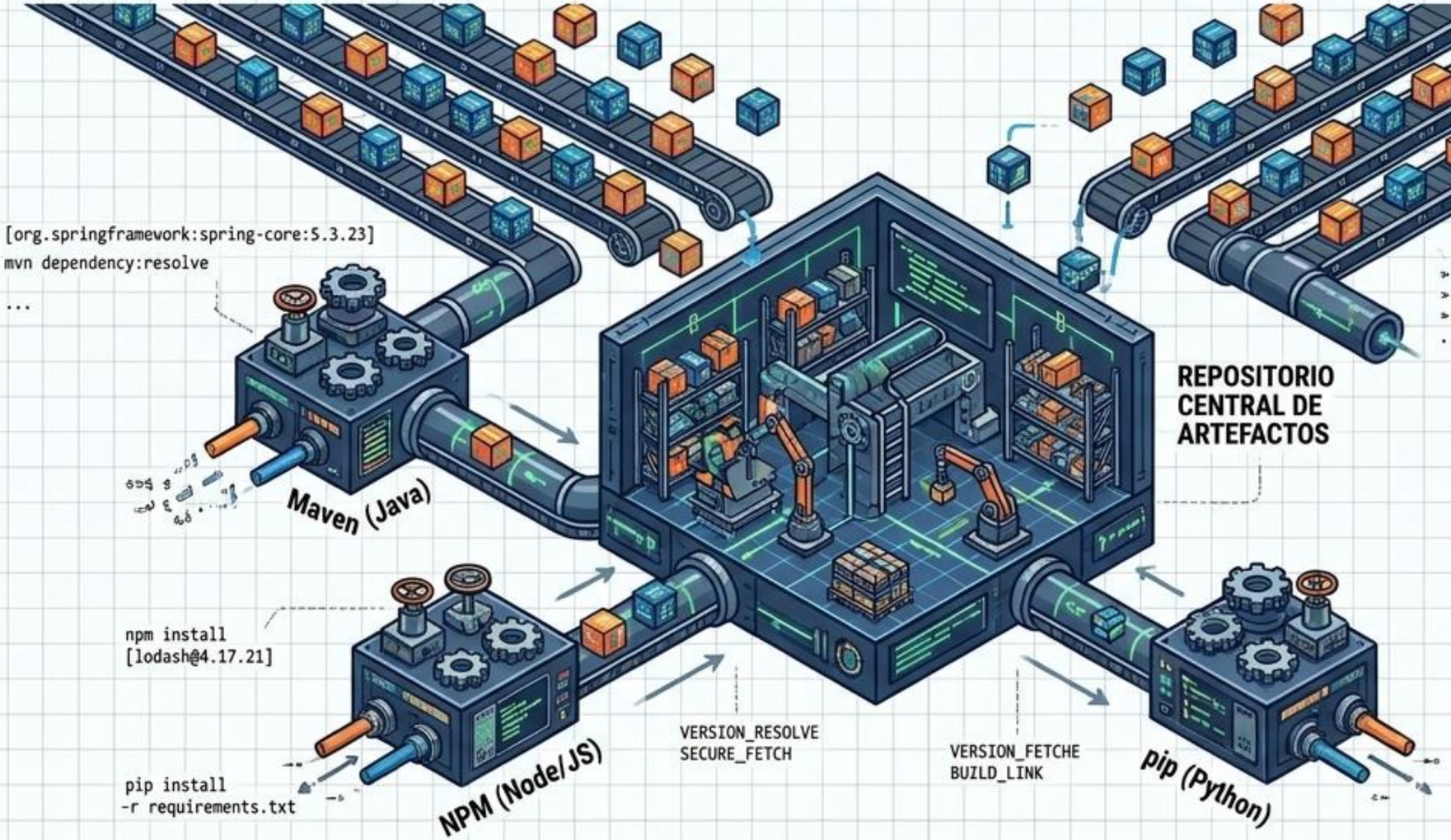
El Paradigma del Reúso: Dos Caras de la Moneda

	Construcción PARA el Reúso (Crear)	Construcción CON Reúso (Consumir)
Enfoque	Diseñar librerías genéricas, modulares y agnósticas al contexto del cliente.	Integrar frameworks externos y componentes de terceros al dominio específico.
Reto Arquitectónico	Lograr generalidad absoluta sin caer en la sobre-ingeniería.	Gestionar el acoplamiento agresivo y la inversión de control (IoC).
Artefacto Típico	**Librerías base** (ej. utilitarios matemáticos, parsers independientes).	Frameworks pesados y prescriptivos (ej. Spring, React, Django).

© 2024 Premium Engineeri...
Gemini Notebook

El Pegamento del Reúso: Gestión de Dependencias

Realidad Moderna: 10% código propio, 90% dependencias de terceros.



El Rol del Gestor

Automatización crítica para la resolución de versiones, descarga segura y vinculación del código externo en el tiempo de construcción.

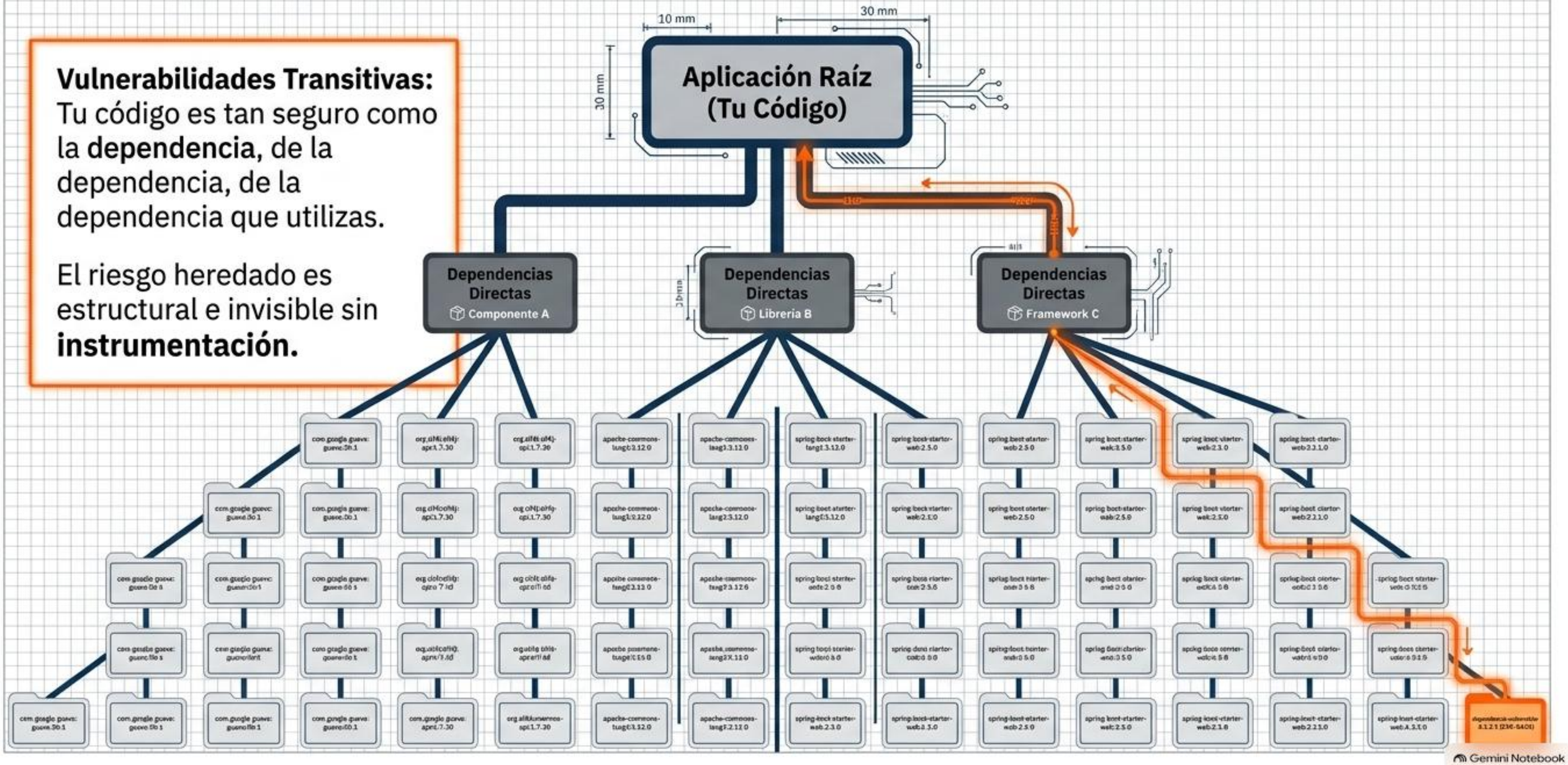
© 2024 Premium Engineering Systems

Gemini Notebook

La Cadena de Suministro de Software (Supply Chain)

Vulnerabilidades Transitivas:
Tu código es tan seguro como la **dependencia**, de la dependencia, de la dependencia que utilizas.

El riesgo heredado es estructural e invisible sin **instrumentación.**



Gemini Notebook

Riesgos y Responsabilidades en el Ecosistema



Riesgos de Seguridad

Inyecciones de código malicioso en repositorios públicos, secuestro de paquetes (typosquatting) y puertas traseras ocultas en actualizaciones menores.

`[threat: code_injection | typo: squatting | backdoor: hidden_updates]`



Conflictos de Licencias

Incompatibilidad legal silenciosa oculta en el árbol de dependencias (ej. código Copyleft GPL contaminando la distribución de software comercial cerrado).

`[license: conflict | copyleft: gpl_contamination | proprietary: software]`



Estabilidad y Rompimiento

El efecto dominó de los parches de seguridad y las actualizaciones incompatibles, llevando al colapso del sistema (dependency hell).

`[stability: collapse | dependency: hell | patch: incompatible]`

© 2024 Premium Engineering Systems

Gemini Notebook

Metodología SW-605: Ingeniería Continua

01 Teoría + Práctica Intensiva

Conexión directa entre conceptos arquitectónicos abstractos y casos reales de la industria del software.

`[theory: abstract | practice: real_world | bridge: established]`

02 Desarrollo Incremental

Iteración semanal rigurosa enfocada en la construcción sólida, instrumentación, depuración avanzada y refactorización implacable del código.

`[cycle: weekly | focus: robust_build | debug: advanced | refactor: ruthless]`

03 El Desafío Final

El curso no culmina creando algo nuevo desde cero, sino enfrentando la realidad profesional: evolución técnica e ingeniería reversa de un sistema legacy existente.

`[challenge: legacy_evolution | skill: reverse_engineering | reality: professional]`

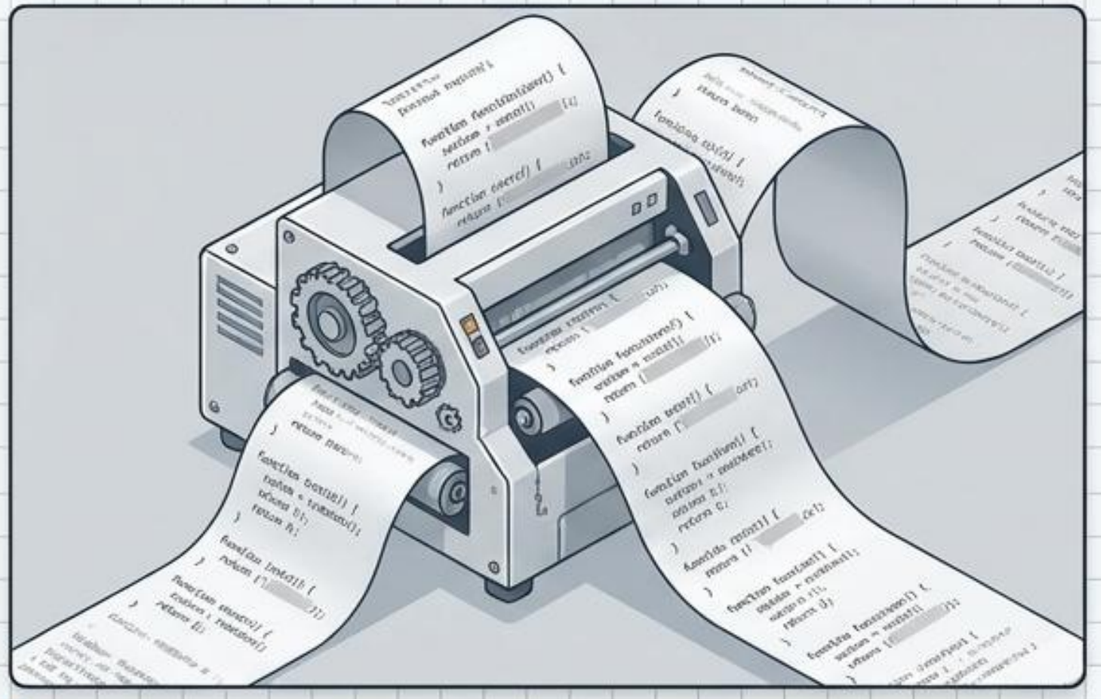


© 2024 Premium Engineering Systems

Gemini Notebook

IA Asistida: Ética, Código y Responsabilidad

La Realidad: Las herramientas de IA (Copilot, ChatGPT) actúan como aceleradores tácticos de sintaxis. Generan volumen, pero **carecen de contexto arquitectónico**.



`[focus: syntax_acceleration | volume: high | context: absent]`

El Mandato del Ingeniero: La IA no asume responsabilidad legal, de seguridad ni de diseño. **Tú eres el arquitecto del runtime.**



`[responsibility: human | design: architect | runtime: ownership]`

Tu Aporte Inremplazable: Razonar sobre el comportamiento dinámico, analizar los trade-offs de robustez vs. rendimiento, y auditar implacablemente la calidad final.

© 2024 Premium Engineering Systems

Gemini Notebook

Sistema de Evaluación

$$NF = (1*EP + 2*EF + 1*PP) / 4$$

EP: Examen Parcial

Semana 8. Evaluación rigurosa de las Unidades 1 y 2.

EF: Examen Final

Semana 16. Evaluación integral con énfasis en las Unidades 3 y 4.

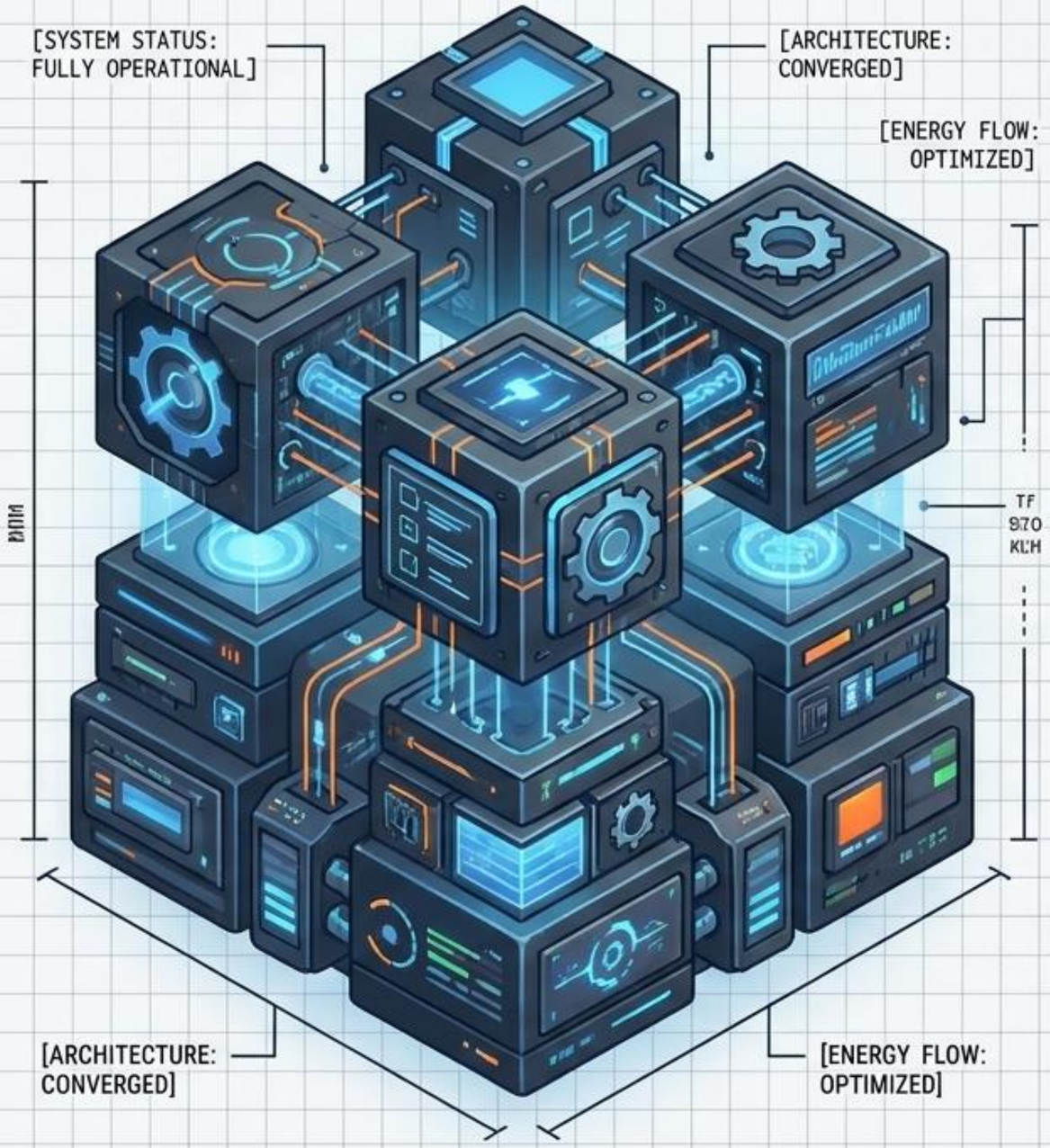
PP: Prácticas Calificadas

Promedio de 4 PCs intensivas.

- PC1: Construcción robusta y contratos.
- PC2: Concurrencia y tolerancia a fallas.
- PC3: Distribución y observabilidad.
- PC4: Reingeniería y evolución *legacy*.

© 2024 Premium Engineering Systems

Gemini Notebook



El Camino por Delante

Síntesis

La construcción de software avanzada es la ingeniería de la supervivencia del código. Exige dominar el tiempo de ejecución, gobernar dependencias de terceros con rigor industrial y verificar incesantemente.

Próxima Semana

Problemas de Tiempo de Ejecución en la Construcción Orientada a Objetos. Profundizaremos en el polimorfismo, la metaprogramación y las implicancias de seguridad del enlace tardío.

© 2024 Premium Engineering Systems

Gemini Notebook